

VOCALDINE

Voice-Powered Restaurant Reservation Platform

Product & Architecture Brief

Business Requirements · Success Criteria · Scope · Data Models · DDD · ADR

Product	VocalDine — voice booking assistant for restaurants	Stack	MERN — MongoDB, Express, React, Node.js
Version	1.0 — Development Prototype	Author	Sandeep Mukku
Date	March 2026	Status	In Development

01 BUSINESS REQUIREMENTS

Problem Statement

Traditional restaurant booking is form-heavy and slow. VocalDine eliminates friction — guests reserve a table through a guided voice conversation. No forms. No phone calls.

Business Objectives

- Reduce booking time from ~3 minutes (web form) to under 60 seconds via voice.
- Capture guest preferences (cuisine, seating, requests) for analytics-driven upsell.
- Enable operators to manage reservations without a third-party POS dependency.
- Use real-time weather to proactively recommend indoor/outdoor seating.
- Auto-send email confirmations to reduce no-shows.

Functional Requirements

ID	Requirement	Priority
FR-01	Accept spoken booking intent; guide user through conversational flow	MUST
FR-02	Parse natural language dates ('next Friday', 'tomorrow')	MUST
FR-03	Fetch real-time weather for booking date and city	MUST
FR-04	Recommend indoor/outdoor seating based on weather condition	MUST
FR-05	Show only available time slots for the chosen date	MUST
FR-06	Prevent double-booking of same date + time slot	MUST
FR-07	Persist bookings in MongoDB with full guest metadata	MUST
FR-08	Send HTML confirmation email after successful booking	MUST
FR-09	Admin portal: list all bookings, delete, export CSV	MUST
FR-10	Admin analytics: total bookings, peak hours, cuisine chart	SHOULD
FR-11	Admin portal password-gated	MUST
FR-12	TTS reads every agent response aloud	MUST

Non-Functional Requirements

Category	Requirement	Target
Performance	API response time P95	< 800ms
Browser	Voice recognition compatibility	Chrome / Chromium
Availability	Uptime during restaurant hours	> 99.5%
Security	Admin route protection	Password-gated (JWT in v2)
Error	Weather API degradation	Graceful fallback

02 SUCCESS CRITERIA

Definition of Done — Sprint Level

- Voice booking completes end-to-end without manual fallback on Chrome.
- No double-booking possible for identical date + time combination.
- Email preview URL logged in console after every successful booking.
- Admin dashboard loads all bookings + analytics in < 2 seconds on localhost.
- CSV export contains all required fields (ID, name, email, guests, date, time, seating, cuisine).

KPIs

KPI	Baseline	Target (Month 3)	How Measured
Voice booking completion rate	N/A (new)	> 75%	Bookings saved ÷ sessions started
Time-to-book via voice	~180s form	< 60s	Session timestamp delta
Double-booking incidents	N/A	0	DB conflict query
Email delivery	N/A	100%	Nodemailer send count
Weather API fallback rate	N/A	< 5%	Error log monitoring

Acceptance Criteria by Epic

Epic	Acceptance Criteria
Voice Booking	User completes full booking (10 steps) using voice alone; TTS reads every response.
Weather	Condition returned for booking date; fallback triggers if API unavailable.
Slot Availability	Available slots exclude confirmed bookings. Fully booked dates prompt alternate date.
Email	HTML email sent post-booking. Preview URL in console. Skip path works.
Admin Analytics	Total bookings, top cuisine, peak hour, bar chart visible on dashboard load.
Conflict Prevention	Duplicate slot returns HTTP 400. No duplicate record created in DB.

03 SCOPE TO DELIVER

IN SCOPE — v1.0	OUT OF SCOPE — Backlog v2
✓ 10-step guided voice booking flow ✓ OpenWeatherMap integration (5-day forecast + geocoding) ✓ MongoDB booking persistence + conflict detection ✓ Nodemailer Ethereal confirmation email ✓ Admin dashboard — list, delete, analytics, CSV export ✓ Password-gated admin portal ✓ RESTful Express API (7 endpoints) ✓ React SPA with cyberpunk dark UI theme	✗ OpenAI / LLM free-form conversation ✗ JWT / OAuth2 admin authentication ✗ Twilio SMS / WhatsApp confirmation ✗ Multi-restaurant / multi-location ✗ Hinglish voice recognition ✗ Live POS / table management integration ✗ Mobile app (iOS / Android) ✗ Automated test suite (unit / E2E)

Delivery Roadmap

Sprint	Duration	Deliverables
Sprint 0	1 wk	Repo, DB config, Express skeleton, React CRA scaffold
Sprint 1	2 wk	Booking model + CRUD API, conflict detection, available-slots endpoint
Sprint 2	2 wk	Voice flow (steps 0–7), TTS, chrono-node, VoiceAssistant component
Sprint 3	2 wk	Weather integration, seating recommendation, email confirmation
Sprint 4	1 wk	Admin dashboard, Chart.js analytics, delete, CSV export, auth gate
Sprint 5	1 wk	QA, bug fixes, CSS polish, README, deployment prep

04 DATA MODELS

4.1 Conceptual Model

Why: Establishes business entities and relationships in plain language — no technology assumptions. Why not jump to physical: Physical decisions made without a conceptual model lead to schemas that fight the business domain.

Entity	Description	Key Relationships
GUEST	Person making a reservation (anonymous in v1)	Places one or many BOOKINGS
BOOKING	Core transaction — all reservation details	Occupies one TIME SLOT; has one WEATHER snapshot
TIME SLOT	Named period (e.g. 7:00 PM) on a specific date	0 or 1 confirmed BOOKING per date
WEATHER	Forecast snapshot at booking creation time	Belongs to exactly one BOOKING
SEATING PREFERENCE	Indoor or outdoor — influenced by weather	Belongs to exactly one BOOKING
CUISINE PREFERENCE	Guest's stated food type — captured for analytics	Belongs to exactly one BOOKING

4.2 Logical Model — Booking Entity

Why: Defines attributes, types, keys, constraints independent of MongoDB. Why not skip to physical: Logical model catches design errors (e.g. missing uniqueness constraint) before writing code.

Attribute	Type	Constraint	Business Rule
bookingId	String (UUID v4)	PK · UNIQUE · NOT NULL	App-generated — never DB ObjectId exposed to API
customerEmail	String	OPTIONAL · lowercase	Must contain '@' to trigger email dispatch
numberOfGuests	Integer	NOT NULL · min 1	Spoken or typed — textToNumber() parses words to int
bookingDate	Date	NOT NULL	Normalised to midnight UTC before persistence
bookingTime	String (enum)	NOT NULL	Must be one of 7 predefined slots
seatingPreference	Enum	{indoor, outdoor}	Derived from weather condition + user choice
weatherInfo	Embedded Object	OPTIONAL	Immutable snapshot — never updated post-booking
status	Enum	{confirmed,pending,cancelled}	Default: confirmed. Conflict check filters on this
createdAt/updatedAt	Timestamp	Auto	Mongoose timestamps: true

Key Logical Constraints — why each exists:

- **UNIQUE(bookingDate + bookingTime + status='confirmed')** — why: prevents two guests booking the same slot. What if missing: race condition allows double-booking under concurrent requests.
- **weatherInfo embedded (not normalised)** — why: it is queried only with its parent booking, never independently. What if normalised: adds a JOIN equivalent (populate) for zero query benefit.
- **bookingDate normalised to midnight** — why: date-only comparison in MongoDB requires consistent time component. What if skipped: findOne on date returns inconsistent results across timezones.

4.3 Physical Model — MongoDB Collection: bookings

Why MongoDB: variable-shape weatherInfo sub-document, native \$group aggregation, zero-migration schema evolution. Why not PostgreSQL: weatherInfo forces a JOIN or JSONB column for zero relational benefit.

Field	BSON Type	Index	Physical Note
_id	ObjectId	AUTO PRIMARY	MongoDB internal — not exposed in API responses
bookingId	String	UNIQUE INDEX	All API routes use this — not _id

Field	BSON Type	Index	Physical Note
bookingDate	Date	COMPOUND *	Always midnight UTC
bookingTime	String	COMPOUND *	Plain string match against 7-slot list
status	String	COMPOUND *	Compound index enables conflict check in $O(\log n)$
cuisinePreference	String	—	Used in \$group aggregation pipeline only
weatherInfo	Object	—	Embedded: {temperature: Double, condition: String, description: String}

* Recommended compound index: { bookingDate:1, bookingTime:1, status:1 } — why: serves both the conflict-check `findOne()` and `available-slots find()` in a single index scan. What if missing: full collection scan on every booking creation = $O(n)$ performance.

Analytics Aggregation Pipelines (Physical):

Query	Pipeline	Output
Peak Hours	\$group by bookingTime → \$sum:1 → \$sort:count desc	[{ _id:'7:00 PM', count:N }]
Popular Cuisines	\$group by cuisinePreference → \$sum:1 → \$sort desc	[{ _id:'Indian', count:N }]
Total Bookings	countDocuments()	Integer

05 HIGH-LEVEL APPLICATION COMPONENTS — DOMAIN DRIVEN DESIGN

Bounded Contexts — why DDD here:

Why: VocalDine has three distinct business concerns that should not bleed into each other. Why not a single 'booking' module: mixing weather logic with email logic with reservation logic creates a God Object that is impossible to test or replace independently.

Bounded Context	Type	Owns	Implementation Location
Reservation	Core Domain	Booking lifecycle, conflict detection, slot availability	booking.controller.js + booking.model.js
Weather	Supporting Domain	Forecast fetch, seating recommendation logic	config/weather.js + weather.route.js
Notification	Generic Subdomain	Email composition and dispatch	booking.controller.js (sendConfirmationEmail)
Analytics	Supporting Domain	Aggregation queries, dashboard data	booking.controller.js (getAnalytics)
Identity & Access	Generic Subdomain	Admin authentication guard	AdminDashboard.js (frontend, v2: JWT)

DDD Building Blocks — Reservation Context

Building Block	Component	Why this pattern
Aggregate Root	Booking	Owns all booking state. Enforces invariants (status enum, min guests). Single unit of persistence.
Value Object	BookingTime (string)	Immutable. Equality is structural — '7:00 PM' == '7:00 PM'. No identity needed.
Value Object	WeatherInfo (embedded obj)	Immutable snapshot at creation time. Never updated. Queried only via parent Booking.
Value Object	SeatingPreference (enum)	indoor/outdoor — no identity, purely descriptive. Cannot exist without a Booking.
Domain Service	ConflictChecker (findOne)	Operates on Booking aggregate but belongs to no single instance — cross-aggregate uniqueness check.
Domain Service	SlotFilter (getAvailableSlots)	Derives availability by subtracting takenSlots from masterList. Pure function, no side effects.
Repository	Mongoose Booking model	Abstracts MongoDB — find, findOne, save, findOneAndDelete. Controller never writes raw queries.
Application Svc	BookingController	Orchestrates: validate → conflict check → fetch weather → persist → dispatch email.
Domain Event*	BookingConfirmed (implicit)	*Should be explicit in v2. Currently implicit — email triggered synchronously after save().

Frontend Component Architecture

Layer	Component	Responsibility / Why isolated
Page/Container	Home.js	Owns 10-step state machine. Why: all booking state lives here — child components are stateless.
Page/Container	AdminDashboard.js	Owns admin state + auth guard. Why: separate page = separate bounded context in the UI.
UI Component	AgentBubble.js	Renders agent message. Why pure presentational: zero logic = zero bugs, easy to restyle.
UI Component	VoiceAssistant.js	Wraps react-speech-recognition. Why isolated: voice hardware concern separated from booking logic.
UI Component	BookingSummary.js	Pre-confirmation summary table. Why isolated: same component reusable in email template in v2.
UI Component	LoadingIndicator.js	Animated loading dots. Why isolated: async feedback decoupled from what is loading.

Layer	Component	Responsibility / Why isolated
Service Layer	services/api.js	Axios instance + named functions. Why: UI components never import axios directly — swap to fetch in v2 by editing one file.

Conversational State Machine — 10-step flow in Home.js

Why a state machine: the booking flow is strictly linear with one active step at a time. Why not Redux/Zustand: local useState is sufficient — state does not need to be shared across routes.

Step	State Name	Input Type	Output / Side Effect
-1	Intent Detection	Voice	Keyword match (book/reservation/table) → step 0
0	Guest Count	Voice	textToNumber() → store numberOfGuests → step 1
1	Date + Weather	Voice	chrono-node → parallel API: getWeather + getSlots → step 2
2	Seating	Voice	indoor/outdoor keyword → store seatingPreference → step 3
3	Time Slot	Voice	normalizeTime() + validate vs availableSlots → step 4
4	Cuisine	Voice	Free text → store cuisinePreference → step 5
5	Email	Typed	Typed input or skip → store customerEmail → step 6
6	Special Requests	Voice	Free text → store specialRequests → step 7
7	Confirmation	Voice	'yes' → createBooking() API call → success / 'no' → reset()

06 TECH STACK — ARCHITECTURAL DECISION RECORDS

Decision Framework: Every choice is evaluated against three VocalDine-specific constraints — (1) voice-first UX demands zero STT latency, (2) single-developer velocity demands minimal boilerplate, (3) zero infrastructure cost at prototype stage.

ADR-001 MongoDB + Mongoose as Database		ACCEPTED
Why Chosen	MongoDB beats PostgreSQL for VocalDine because the booking document contains an embedded variable-shape weatherInfo object. PostgreSQL forces a JOIN (separate weather table) or JSONB column for zero relational benefit. Native \$group aggregation serves analytics in 3 lines vs raw SQL.	
Why Rejected	PostgreSQL — adds migrations, ORM, and JOIN overhead for a schema with no relational relationships. Firebase — vendor lock-in, aggregation requires paid Cloud Functions.	
Risk / v2	No atomic conflict check — findOne + save is not a transaction. Mitigate in v2 with findOneAndUpdate + upsert or MongoDB transactions.	
ADR-002 React 19 (CRA) as Frontend Framework		ACCEPTED
Why Chosen	react-speech-recognition is React-exclusive — it provides useSpeechRecognition hook, transcript, listening state, and lifecycle management as reactive values. Vue and Svelte have no equivalent: manual Web Speech API wiring adds ~120 lines of error-prone event handling. useState hooks map cleanly to the 10-step state machine.	
Why Rejected	Vue 3 — no mature Web Speech wrapper; manual integration required. Next.js — SSR overhead + 'use client' guards everywhere for a booking widget with no indexable URLs. Svelte — zero voice library ecosystem.	
Risk / v2	CRA is in maintenance mode. Migrate to Vite in v2 Sprint 1 — 2-hour migration, zero application code changes.	
ADR-003 Express.js v5 as Backend Framework		ACCEPTED
Why Chosen	7 endpoints need the least ceremony possible. Express writes a booking CRUD endpoint in one file with direct Mongoose calls. v5 natively propagates async errors to middleware — no express-async-errors patch. Unopinionated structure lets routes mirror DDD bounded contexts exactly.	
Why Rejected	NestJS — requires module/controller/service/DTO/schema files before writing a single business logic line. Correct for 50-endpoint enterprise APIs; engineering theater at this scale. Django — introduces Python runtime; entire JS/Mongoose stack becomes unusable.	
Risk / v2	No built-in input validation. Add express-validator or Joi in v2 for all POST endpoints.	
ADR-004 Web Speech API via react-speech-recognition (STT + TTS)		ACCEPTED
Why Chosen	Real-time streaming transcript — words appear as the user speaks, 0ms latency. Zero cost. No API key. SpeechSynthesisUtterance supports en-IN locale. The conversational loop (speak → respond → speak) must feel instant — any API-based STT introduces 500ms–2s dead silence that destroys the UX.	
Why Rejected	OpenAI Whisper — 500ms–2s latency per utterance kills conversational feel. Google Cloud STT — streaming available but requires WebSocket backend proxy + paid tier; identical Google processing to Web Speech API at higher engineering cost.	
Risk / v2	Chrome/Chromium only. Add typed fallback for all voice steps in v2 to support Firefox/Safari users.	
ADR-005 chrono-node v2 for Natural Language Date Parsing		ACCEPTED
Why Chosen	Users say 'next Saturday', 'tomorrow', 'this weekend' — not '2026-03-14'. Date.parse() returns NaN for all relative expressions. chrono-node handles the full range, returns null on failure (clean re-prompt trigger), and has 5 years of edge-case handling baked in.	
Why Rejected	Date.parse() — fails on every relative date expression. Moment.js — deprecated since 2020, requires plugin for relative dates, larger bundle. Custom regex — brittle, misses edge cases like 'next Monday when today is Monday'.	
Risk / v2	English-only in v1. Hinglish date expressions unsupported until v2 OpenAI integration.	

ADR-006 Nodemailer + Ethereal SMTP (dev) → SendGrid (prod)		ACCEPTED (dev stage)
Why Chosen	Ethereal auto-creates a test SMTP account on server start — zero config, zero cost, zero account setup. Preview URL logged to console for verification. Production swap is 3 config lines in the transporter object (host, user, pass). Identical Nodemailer API throughout.	
Why Rejected	SendGrid (dev) — requires account, domain verification, API key, sender identity approval. AWS SES — IAM setup + sandbox approval takes 24–48 hours. Completely disproportionate for prototype stage.	
Risk / v2	Ethereal emails are NOT delivered to real inboxes. Production deployment requires transporter swap to SendGrid/Resend before go-live.	

ADR-007 OpenWeatherMap (Geocoding + 5-day Forecast APIs)		ACCEPTED
Why Chosen	Two-call chain (city → lat/lon → forecast) is 10 lines of Axios. Free tier: 60 calls/min, 1000/day. weather[0].main returns 'Rain'/'Clear'/'Clouds' — maps to seating recommendation with a single string.includes() check. Proxied through Express backend so API key never reaches frontend bundle.	
Why Rejected	WeatherAPI.com — 3-day free tier insufficient for booking horizon. Open-Meteo — no geocoding; requires a third API dependency + WMO integer lookup table. AccuWeather — 50 calls/day free limit is too low for development.	
Risk / v2	5-day forecast limit — bookings beyond 5 days fall back to current weather. Geo call can be cached in v2 (Mumbai coordinates never change) to halve API usage.	

Complete Stack Scorecard

Layer	Chosen	Runner-up	Decisive Reason Chosen Wins
Database	MongoDB	PostgreSQL	Embedded docs + native aggregation vs JOIN overhead for zero relational benefit
Frontend	React 19	Vue 3	react-speech-recognition is React-exclusive — no Vue equivalent
Backend	Express v5	NestJS	7 endpoints need zero ceremony — NestJS adds 6 files before business logic
STT/TTS	Web Speech API	Whisper API	0ms streaming vs 500ms–2s API latency — conversational UX requires instant response
Date Parsing	chrono-node v2	Date.parse()	Date.parse('next Saturday') = NaN — full stop
Email	Nodemailer+Ethereal	SendGrid	Zero config for prototype; identical API means 3-line swap to production
Weather	OpenWeatherMap	WeatherAPI.com	Free 5-day forecast + geocoding in one provider; condition field maps directly to seating logic

No choice was made because it was popular. Every choice has a named competitor it beats on a specific, measurable criterion relevant to VocalDine's constraints.